

DEPLOYMENT.md — Деплой на Vercel и настройка Supabase

Этот документ — пошаговая инструкция по выкладке приложения EVA в продакшен. Делается **один раз при инициализации** и потом обновляется только при изменении инфраструктуры.

0. Контекст

В проде у нас три инфраструктурных компонента:

1. **Vercel** — хостит Next.js приложение (страницы webapp + API роуты).
2. **Supabase** — Postgres БД + Edge Functions (для отложенных задач и реакции на webhooks).
3. **Telegram Bot API** — webhook от бота смотрит на наш Vercel-домен.

Между ними есть несколько связей:

- Supabase Database Webhook → Edge Function (на каждый INSERT в `test_results`).
 - Supabase pg_cron → Edge Function (раз в минуту обрабатывает `bot_tasks_queue`).
 - Telegram Bot → Vercel `/api/webhook/telegram` (на каждое сообщение/callback).
-

1. Vercel — деплой Next.js

1.1. Подготовка репозитория

В корне репозитория должны быть:

- `package.json` с командами `build` и `start`.
- `next.config.js` или `next.config.ts` с актуальными настройками.
- Все исходники запущены в `main` ветку GitHub.

1.2. Создание проекта в Vercel

1. Зайти на vercel.com, залогиниться через GitHub.
2. `Add New...` → `Project` → выбрать репозиторий EVA.
3. Framework Preset → `Next.js` (автодетектится).
4. Root Directory → `./` (если репо плоский).
5. Build Command → `npm run build` (по умолчанию).
6. Output Directory → `.next` (по умолчанию).

7. **Не нажимать `Deploy` сразу!** Сначала добавить env-переменные — см. п. 1.3.

1.3. Environment Variables

В Vercel → `Project Settings` → `Environment Variables` добавь следующие переменные. Для каждой выбери все три окружения: **Production** / **Preview** / **Development** (или только Production, если тебе нужен отдельный preview без эффекта).

```
# === Supabase ===
NEXT_PUBLIC_SUPABASE_URL           = https://<project-ref>.supabase.co
NEXT_PUBLIC_SUPABASE_ANON_KEY      = <anon ключ>
SUPABASE_SERVICE_ROLE_KEY         = <service_role ключ> [секрет]
SUPABASE_JWT_SECRET               = <jwt secret из Supabase> [секрет]

# === Telegram ===
TELEGRAM_BOT_TOKEN                 = <токен бота> [секрет]
TELEGRAM_BOT_USERNAME              = sprosievubot
TELEGRAM_CHANNEL_ID               = -100xxxxxxxxxxx
TELEGRAM_CHANNEL_URL              = https://t.me/sprosievu
TELEGRAM_WEBHOOK_SECRET           = <длинная случайная строка> [секрет]

# === Публичные URL ===
NEXT_PUBLIC_APP_URL               = https://eva-9udm.vercel.app
NEXT_PUBLIC_BOT_USERNAME          = sprosievubot
NEXT_PUBLIC_TELEGRAM_CHANNEL_URL  = https://t.me/sprosievu

# === Админка ===
ADMIN_PIN                         = 0999 [секрет]

# === Внутренний JWT ===
EVA_JWT_SECRET                   = <длинный случайный секрет> [секрет]

# === Видео-подарок ===
GIFT_VIDEO_FILE_ID               = <пусто пока, потом обновить>
```

Важные замечания:

- `NEXT_PUBLIC_APP_URL` — **без слэша на конце**. Иначе склейки типа `${URL}/start.png` дадут двойной слэш.
- Если хочешь, чтобы URL автоподставлялся под текущий деплой Vercel (включая preview-домены) — можно использовать `process.env.VERCEL_URL` в коде как fallback:

ts

```
export const getAppUrl = () =>
  process.env.NEXT_PUBLIC_APP_URL ||
  (process.env.VERCEL_URL ? `https://${process.env.VERCEL_URL}` : 'http://localhost')
```

- Все секретные переменные **не должны** иметь префикс `NEXT_PUBLIC_*` — иначе они утекут в браузер.
- Если меняешь env-переменную — нужно сделать **Redeploy** проекта (Vercel не подхватывает на лету).

1.4. Первый деплой

После настройки env нажать `Deploy`. Дождаться сборки (обычно 1–3 мин). По завершении Vercel выдаст `https://eva-<hash>.vercel.app`.

1.5. Кастомный домен (опционально)

Если хочешь свой домен (типа `app.sprosievu.com`):

1. `Project Settings` → `Domains` → `Add`.
2. Ввести домен → Vercel покажет, какие DNS-записи поставить.
3. Поставить записи у регистратора домена.
4. Дождаться валидации (5–60 минут).
5. **Обновить** `NEXT_PUBLIC_APP_URL` в env на новый домен.
6. Сделать `Redeploy`.

1.6. Промоушен Production

В Vercel есть Production и Preview деплои. Production — это тот, что соответствует ветке `main` и доступен по основному домену. Preview — для каждой PR-ветки.

После деплоя `main` → автоматически становится Production. Для пробного релиза можно сделать `Promote to Production` из меню деплоя.

2. Supabase — настройка

2.1. Применение миграций

После деплоя кода нужно применить миграции 107 и 108 (плюс все предыдущие, если БД пустая).

Способ 1 — через Supabase CLI (рекомендуется):

```
bash
```

```
# В корне репозитория
supabase link --project-ref <project-ref>
supabase db push
```

`db push` применит все миграции из `supabase/migrations/`, которых ещё нет на сервере.

Способ 2 — через SQL Editor (если CLI недоступен):

1. Открыть Supabase Dashboard → `SQL Editor`.
2. Скопировать содержимое `107_add_mixed_trait_sent_flag.sql` → выполнить.
3. Скопировать содержимое `108_extend_submit_test_rpc_with_cooldown.sql` → выполнить.
4. Проверить: `SELECT column_name FROM information_schema.columns WHERE table_name = 'profiles' AND column_name LIKE '%mixed%';` — должно вернуть `mixed_trait_sent` и `mixed_trait_sent_at`.

2.2. Database Webhooks

Нужны для триггера Edge Function `process-bot-notifications` при INSERT/UPDATE в `test_results`.

1. Supabase Dashboard → `Database` → `Webhooks` → `Create a new hook`.
2. Заполни:
 - **Name:** `notify-on-test-result`
 - **Table:** `test_results`
 - **Events:** ☒ Insert, ☒ Update
 - **Type:** Supabase Edge Functions
 - **Edge Function:** `process-bot-notifications`
 - **HTTP Headers:** автоматически добавится `authorization: Bearer <service_role_key>`.
3. Сохранить.

Аналогично, если в LOGIC.md решено отправлять смешанную опору при UPDATE `profiles.invites_count`, можно создать второй webhook на `profiles`. Но проще: это делает Edge Function `process-bot-queue` через `check_referral_12h` или watchdog в `process-bot-notifications`.

2.3. Edge Functions — деплой

Все Edge Functions лежат в `supabase/functions/<name>/index.ts`. Чтобы задеплоить:

```
bash
```

```
supabase functions deploy process-bot-queue --project-ref <project-ref>
supabase functions deploy process-bot-notifications --project-ref <project-ref>
supabase functions deploy send-periodic-reminders --project-ref <project-ref>
```

После деплоя в Dashboard → **Edge Functions** появятся 3 функции.

2.4. Секреты Edge Functions

Edge Functions работают в отдельном окружении и не видят env-переменные Vercel. Им нужны свои секреты:

```
bash

supabase secrets set TELEGRAM_BOT_TOKEN=<токен> --project-ref <project-ref>
supabase secrets set TELEGRAM_CHANNEL_ID=-100xxxxxxxxxx --project-ref <project-ref>
supabase secrets set NEXT_PUBLIC_APP_URL=https://eva-9udm.vercel.app --project-ref <project-ref>
supabase secrets set NEXT_PUBLIC_BOT_USERNAME=sprosievubot --project-ref <project-ref>
supabase secrets set GIFT_VIDEO_FILE_ID="" --project-ref <project-ref>
```

Также Edge Functions автоматически имеют доступ к:

- **SUPABASE_URL** — URL проекта.
- **SUPABASE_ANON_KEY** — публичный ключ.
- **SUPABASE_SERVICE_ROLE_KEY** — service_role ключ.

Их **не нужно** дополнительно прописывать через **supabase secrets set** — они доступны из коробки.

Проверить установленные секреты:

```
bash

supabase secrets list --project-ref <project-ref>
```

2.5. pg_cron — расписание для очереди

Чтобы **process-bot-queue** запускалась автоматически каждую минуту, нужно настроить **pg_cron** (миграция 105 это делает, но если она не применилась — выполни вручную):

В SQL Editor:

```
sql
```

```

-- 1. Включить расширение pg_cron (если ещё не включено)
CREATE EXTENSION IF NOT EXISTS pg_cron;

-- 2. Включить pg_net для HTTP-запросов из БД
CREATE EXTENSION IF NOT EXISTS pg_net;

-- 3. Создать (или пересоздать) задание
SELECT cron.unschedule('process-bot-queue-every-minute')
  WHERE EXISTS (SELECT 1 FROM cron.job WHERE jobname = 'process-bot-queue-every-minute');

SELECT cron.schedule(
  'process-bot-queue-every-minute',
  '* * * * *', -- каждую минуту
  $$
SELECT net.http_post(
  url := 'https://<project-ref>.supabase.co/functions/v1/process-bot-queue',
  headers := jsonb_build_object(
    'Content-Type', 'application/json',
    'Authorization', 'Bearer <SERVICE_ROLE_KEY>'
  ),
  body := jsonb_build_object('action', 'process_queue')
);
  $$
);

-- 4. Проверить, что задание создано
SELECT * FROM cron.job;

```

Подставь `<project-ref>` и `<SERVICE_ROLE_KEY>` своими значениями.

SERVICE_ROLE_KEY никогда не должен попадать в публичный репозиторий — этот SQL выполняется только в защищённом SQL Editor Supabase.

2.6. Дополнительный cron для `send-periodic-reminders` (fallback)

Раз в сутки (на случай, если `bot_tasks_queue` сбойнул и `cooldown_reminder` не сработал):

```
sql
```

```
SELECT cron.schedule(  
  'send-periodic-reminders-daily',  
  '0 9 * * *', -- каждый день в 09:00 UTC  
  $$  
  SELECT net.http_post(  
    url := 'https://<project-ref>.supabase.co/functions/v1/send-periodic-reminders',  
    headers := jsonb_build_object(  
      'Content-Type', 'application/json',  
      'Authorization', 'Bearer <SERVICE_ROLE_KEY>'  
    ),  
    body := '{}'::jsonb  
  );  
  $$  
);
```

3. Telegram — настройка бота

3.1. Установка webhook

После того, как Vercel задеплоен и доступен:

```
bash  
  
curl -X POST "https://api.telegram.org/bot<TELEGRAM_BOT_TOKEN>/setWebhook" \  
-H "Content-Type: application/json" \  
-d '{  
  "url": "https://eva-9udm.vercel.app/api/webhook/telegram",  
  "secret_token": "<TELEGRAM_WEBHOOK_SECRET>",  
  "allowed_updates": ["message", "callback_query", "my_chat_member"]  
'
```

Должен вернуться `{"ok":true,"result":true,"description":"Webhook was set"}`.

Проверить:

```
bash  
  
curl "https://api.telegram.org/bot<TELEGRAM_BOT_TOKEN>/getWebhookInfo"
```

В ответе должно быть:

- `"url": "https://eva-9udm.vercel.app/api/webhook/telegram"`
- `"pending_update_count": 0`
- `"last_error_message"` — отсутствует или старая.

3.2. Установка Menu Button

В коде есть `/api/bot/setup` — он умеет сам проставить Menu Button. Дёрни его:

```
bash

curl -X POST https://eva-9udm.vercel.app/api/bot/setup \
  -H "Authorization: Bearer <ADMIN_TOKEN>"
```

Если этого роута нет или он не работает — ставим вручную:

```
bash

curl -X POST "https://api.telegram.org/bot<TELEGRAM_BOT_TOKEN>/setChatMenuButton" \
  -H "Content-Type: application/json" \
  -d '{
    "menu_button": {
      "type": "web_app",
      "text": "🍃 Тест",
      "web_app": {"url": "https://eva-9udm.vercel.app"}
    }
  }'
```

3.3. Установка команд бота

```
bash

curl -X POST "https://api.telegram.org/bot<TELEGRAM_BOT_TOKEN>/setMyCommands" \
  -H "Content-Type: application/json" \
  -d '{
    "commands": [
      {"command": "start", "description": "Начать работу с ботом"},
      {"command": "test", "description": "▶ Пройти тест"}
    ]
  }'
```

3.4. Установка картинки и описания бота (опционально)

Через `@BotFather`:

- `/setdescription` → ввести длинное описание для карточки бота.
 - `/setabouttext` → короткое описание (отображается под аватаркой).
 - `/setuserpic` → загрузить аватар.
-

После того как всё задеплоено:

1. Открыть Telegram, зайти в личку бота `@sprosievubot` со своего аккаунта `@evapatrakhina`.
2. Отправить **видео-подарок** прямо в чат.
3. В ответ бот пришлёт ☒ `FILE ID: <строка>`.
4. Скопировать строку.
5. Vercel → `Project Settings` → `Environment Variables` → `GIFT_VIDEO_FILE_ID` → ввести скопированное значение.
6. Supabase → `supabase secrets set GIFT_VIDEO_FILE_ID="<строка>" --project-ref <project-ref>`.
7. **Redeploy** Vercel-проекта (с новой env).
8. Перевыкатить Edge Function `process-bot-queue`: `supabase functions deploy process-bot-queue`.

После этого вместо текстовой заглушки «🎁 Твой подарок готов!» будет приходить настоящее видео с `protect_content: true`.

5. Чек-лист готовности к продакшен-релизу

5.1. Vercel

- ☐ Все env-переменные из п. 1.3 установлены.
- ☐ Деплой прошёл без ошибок (зелёная галочка в Vercel Dashboard).
- ☐ Открыть Production URL в браузере → нет 500-ошибок.
- ☐ `NEXT_PUBLIC_APP_URL` совпадает с реальным Production URL.

5.2. Supabase

- ☐ Миграции 107 и 108 применены.
- ☐ 3 Edge Function задеплоены: `process-bot-queue`, `process-bot-notifications`, `send-periodic-reminders`.
- ☐ Database Webhook на `test_results` (Insert + Update) → `process-bot-notifications`.
- ☐ pg_cron задание `process-bot-queue-every-minute` активно (`SELECT * FROM cron.job;`).
- ☐ `supabase secrets list` показывает все нужные секреты.

5.3. Telegram

- ☐ `getWebhookInfo` показывает корректный URL и `pending_update_count: 0`.
- ☐ Menu Button бота указывает на webapp URL.

- ☐ Команды `/start` и `/test` отображаются в меню бота.

5.4. Smoke-тест на продакшене

Пройти **Сценарий 6.1** из `LOCAL_DEV.md` (полный путь нового пользователя) на проде. Если всё работает — релиз готов.

5.5. Постпродакшен

- ☐ Получить `GIFT_VIDEO_FILE_ID` (см. п. 4) и обновить env в Vercel + Supabase secrets.
 - ☐ Замониторить первые 24 часа — наблюдать за `bot_tasks_queue` (нет ли застрявших задач со `status='failed'`).
 - ☐ Раз в неделю проверять Edge Functions logs на наличие ошибок.
-

6. Откат и аварийные процедуры

6.1. Откатить деплой Vercel

Vercel Dashboard → `Deployments` → найти предыдущий стабильный деплой → `... → Promote to Production`. Происходит за 5 секунд, без пересборки.

6.2. Отключить webhook бота

Если webhook начал слать тонны ошибок, временно отключи:

```
bash

curl -X POST "https://api.telegram.org/bot<TELEGRAM_BOT_TOKEN>/deleteWebhook"
```

После починки — установи заново (см. п. 3.1).

6.3. Остановить cron в Supabase

Если `process-bot-queue` заикливается:

```
sql

SELECT cron.unschedule('process-bot-queue-every-minute');
```

Включить обратно — повторить SQL из п. 2.5.

6.4. Очистить очередь задач

Если в `bot_tasks_queue` накопилось много `failed`-задач:

```
sql
```

```
DELETE FROM bot_tasks_queue WHERE status = 'failed';  
-- или мягче:  
UPDATE bot_tasks_queue SET status = 'cancelled' WHERE status = 'failed';
```

6.5. Резервная копия БД

Supabase делает автоматический бэкап ежедневно (Pro план). Если что-то пошло совсем плохо:

Dashboard → Database → Backups → выбрать дату → Restore.

⚠ **Внимание:** restore полностью затирает текущее состояние БД.

7. Мониторинг

7.1. Vercel Logs

Dashboard → Deployments → выбрать деплой → Functions → View Function Logs. Здесь видны все запросы к API роутам и их статусы.

7.2. Supabase Edge Function Logs

Dashboard → Edge Functions → выбрать функцию → Logs. Видно все вызовы, их время, статус, ошибки.

7.3. Supabase DB Logs

Dashboard → Database → Logs. Полезно при отладке RPC-функций и триггеров.

7.4. Telegram Bot статистика

Через @BotFather → выбрать бота → Bot Settings → Bot Info. Видно количество активных пользователей.

7.5. Простой алерт

Можно завести Healthcheck-сервис (UptimeRobot, Better Uptime) на:

- https://eva-9udm.vercel.app/api/auth (POST с тестовым initData → 401 — это OK, главное не 500).
- https://<project-ref>.supabase.co/functions/v1/process-bot-queue (POST → 401 — OK).

Если запрос отвечает 5xx → алерт.

8. Чеклист для нового деплоя (когда меняем код)

- ☐ Все изменения запущены в `main`.
 - ☐ Vercel автоматически собрал новый деплой.
 - ☐ Если изменены edge-функции → `supabase functions deploy <name>`.
 - ☐ Если изменены миграции → `supabase db push`.
 - ☐ Если изменены env-переменные → обновить в Vercel и в `supabase secrets`.
 - ☐ Smoke-тест: пройти сценарий 6.1 из `LOCAL_DEV.md` на проде.
 - ☐ Проверить Vercel Function Logs и Supabase Edge Function Logs на отсутствие ошибок в первые 5 минут.
-

Никогда не деплой пятницу вечером.